

RP2350 P2P 差動マンチェスタ (BMC) シリアル通信 試作仕様・設計書

完全整数分周 3.125 Mbps / 抵抗レス直結 / TX pio0・RX pio1 分離 / 実験的 PoC 設計

1 システム諸元・概要

Raspberry Pi Pico 2 (RP2350) 同士を外付け抵抗なしのジャンパ線直結 (50cm 以内) で接続し、差動マンチェスタ符号 (BMC) による低遅延な P2P 通信を行う。現行実装では受信 (RX) を pio1 (SM0) で常時リスニング専従とし、送信 (TX) を pio0 (SM0) に配置する。なお、単一ブロック内での動的 TX MUX 共有による多ポート展開は将来構想 (仮説的トポロジ案) である。

項目	諸元値	備考・分類
伝送路	TX, RX, GND (計 3 線直結)	50cm ジャンパ線, 外付け抵抗完全排除
変調方式	Biphase Mark Code (BMC)	クロック埋め込み自己同期, 極性自動追従
ビットレート	3.125 Mbps	公称レート (計算値: $150 \frac{\text{MHz}}{3.0 \times 16}$)
システムクロック	150.0 MHz	標準動作周波数
PIO 分周比	完全整数分周 $\text{clkdiv} = 3.0f$	PIO クロック 50.0 MHz (1 サイクル 20.0 ns, ジッターゼロ)
1 ビット幅 (T_{bit})	非均一: 通常 320.0 ns (16 cyc) / 境界 360.0 ns (18 cyc)	【ソース実装事実】通常 16 サイクル、32bit ワード境界 (OSR 補充) のみ 18 サイクル (+2 サイクル伸長)。RX はエッジ同期で吸収
サンプリング点	境界から 240.0 ns (12 cycles)	$0.75 \times T_{\text{bit}}$ 位置判定 (定常マージン ± 80 ns, SOP 時 $-60/+100$ ns, 机上計算モデル)
PIO リソース消費	TX: 14 / 32 words (pio0), RX: 17 / 32 words (pio1)	【ソース実装事実】TX/RX を独立 PIO ブロックに分離。TX 空き 18 words, RX 空き 15 words
実機計測スループット	157.4 kB/s (15,251 pkts, CRC Err 0, Drops 0)	【ユーザー報告実測値】Pico 2 W / RP2350 150MHz, GP0GP1 直結 (旧シングルレート BMC)。※リファクタリング後ファームウェアは実機検証未実施

2 物理層 (PHY) 仕様

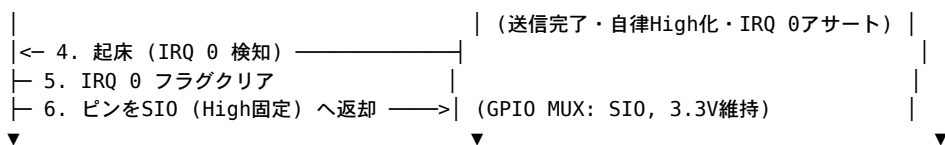
2.1 電気的特性および PAD 設定 (抵抗レス対策)

外付けダンピング抵抗を排除するため、RP2350 の内部 GPIO PAD レジスタによりスルーレートと駆動電流を制限し、50cm 配線上の反射 (リングング) の立ち上がり勾配を緩和する。(※注意: 旧版では「リングングを物理的に抑制し誤エッジゼロ」と記されていたが、任意の配線長・浮遊容量・環境下で「誤エッジゼロ」が成立することは実機で確定された物理的事実ではなく、理論的期待に基づく未検証の前提 [Unverified assumption] である)。

- ・ **TX ピン:** 出力モード / Slew Rate: `GPIO_SLEW_RATE_SLOW` / Drive Strength: `GPIO_DRIVE_STRENGTH_4MA`
- ・ **RX ピン:** 入力モード / `PULL_DOWN` 有効 (内部プルダウン) / Schmitt Trigger 有効

2.2 物理ライン状態定義

状態名	電圧・信号特性	システム動作・条件
未接続 / 切断	0.0 V (Low 固定)	対向ノード未接続 (RX プルダウンによる Link Down)
接続待機 (Idle)	3.3 V (High 固定)	TX 側が High 能動ドライブ (Link Up 即時送信可能)
パケット伝送 (Active)	1.56M~3.125 MHz	BMC 変調パルス (160ns/320ns 反転, 正常デコード)



5 パケット・データリンク層フォーマットおよび再同期設計

5.1 ワイヤフォーマット (Wire Format)

初期設計案では「プリアンプル不要」と主張されていたがこれは誤りであり、任意ビット位相のずれ、起動時過渡現象、ノイズによる誤ビット混入から確実・自動的に復帰するため、ワイヤ先頭に 2 バイトのプリアンプルと 2 バイトの固有同期ワード (Sync Word) を配置するワイヤフォーマットへ改定された。

オフセット	フィールド	サイズ	説明・バリデーション
+0 .. +1	PREAMBLE	2 Bytes	クロック引き込み用交流パターン (0xAAAA = 10101010_10101010b)
+2 .. +3	SYNC	2 Bytes	固有フレーム同期ワード (0x93C7, Big-Endian)
+4	LEN	1 Byte	ペイロード実長 ($1 \leq N \leq 128$)。0 または 129 以上は即時破棄
+5	TYPE	1 Byte	パケット種別 (0x01: Echo Req, 0x02: Echo Res, 0x10: Data, 0x20: Benchmark)
+6	SEQ	1 Byte	シーケンス番号 (0~255 インクリメント、欠落検知用)
+7	RESERVED	1 Byte	ヘッダ側 32bit アライメント用パディング (0x00)
+8 .. +N+7	PAYLOAD	N Bytes	転送実データ (最大 128 バイト)
+N+8 .. +N+9	CRC-16	2 Bytes	CRC-16-CCITT (多項式 0x1021, 初期値 0xFFFF, LEN から PAYLOAD 末尾までを計算)
+N+10 .. +M-1	PADDING	P Bytes	32bit ワード境界・極性パディング ($1 \leq P \leq 4$, $P = 4 - ((N + 6) \bmod 4)$)。先行 $P - 1$ バイトは 0xFF、最終バイトは送信全ビット中の「0」の総数が偶数なら 0xFF、奇数なら 0xFE (0 を 1 個付加して偶数化)。ワイヤ総長 $M = N + 10 + P$ を 4 の倍数化

- ・ **32bit ワード境界アライメントとパディング長:** ポストシンク長 ($N + 6$) バイト (ヘッダ 4B + ペイロード N B + CRC 2B) に対し、パディング長は $P = 4 - ((N + 6) \bmod 4)$ ($1 \leq P \leq 4$ バイト) である (0sim3 バイトの 0x00 ではない)。ワイヤ総長 $M = (N + 10 + P)$ は常に 4 の倍数 ($M \bmod 4 = 0$) となり、受信側 PIO の 32bit autopush で未完了ビットが残存する問題を排除する。
- ・ **偶数反転パリティ補正 (極性整合):** パディング領域のうち先行する $P - 1$ バイトは 0xFF (全ビット 1、ゼロ数 0) で埋められる。最終パディングバイトは、先頭からの送信全バイト (PREAMBLE、SYNC、ヘッダ、ペイロード、CRC) に含まれるビット「0」の総数が既に偶数であれば 0xFF、奇数であれば 0xFE (最下位ビットを 0 にして偶数化) とする。これによりフレーム全体の「0」の総数が常に偶数となり、BMC 変調の数学的性質から最終ビット完了時の物理電位がアイドル電位 (High) と確実に一致する。
- ・ **CRC 計算対象:** ポストシンクデータ (+4 (LEN) から +N+7 (PAYLOAD 末尾) までの全 $N + 4$ バイト) を対象に CRC-16-CCITT (多項式 0x1021, 初期値 0xFFFF) を計算する。先頭の PREAMBLE/SYNC および末尾の PADDING は CRC 対象外である。なお PREAMBLE (0xAAAA) と SYNC (0x93C7) はゼロの総数が偶数個 (PREAMBLE は 8 個のゼロ) であり、フレーミング同期用の固定列として極性パリティを乱さない。
- ・ **送信バッファ形式:** TX DMA で送信する際、バッファ先頭 (第 0 ワード) に PIO のワードカウンタ用ヘッダ ($M / 4 - 1$ (後続データワード数 - 1)) を格納し、続けてフレーム実データを転送する。
- ・ **エンディアン:** 32bit ワード単位で DMA 転送されるが、シリアル線上は先頭バイト +0 (PREAMBLE MSB) から順に MSB-first で送信される。CRC-16 は上位バイト (+N+8)、下位バイト (+N+9) の順。

5.2 ビット単位同期探索と任意ビット位相からの再同期 (Resynchronization)

- ・ **ビットスライド探索:** 受信ドライバ (bmc_driver.c) は、リングバッファ内の受信ビットストリームに対し 1 ビット単位 (bit-by-bit) でスライド走査を行い、同期ワード 0x93C7 (16bit) を照合・検出する。
- ・ **任意ビット位相オフセット耐性:** 線路ノイズや過渡状態によりビットサンプリングが 0~7 ビット任意にずれた場合でも、ドライバはストリーム中から 0x93C7 をビット単位で捕捉してバイト境界を再確立し、次の有効フレームの先頭から即座に正常受信へと復帰する。
- ・ **検証ステータス:** ホスト単体テスト (test_packet.c) において、0~7 ビットの任意オフセット注入、ジッター混入ビットストリーム、および不正フレーム後の次フレーム完全復帰が 100% 検証されている。
- ・ **【重要・実機未検証】:** リファクタリング後の本ファームウェア (PREAMBLE/SYNC bit-by-bit 同期探索) はホストテストで論理検証されたものであり、**実機ハードウェアにおける伝送テストは未実施 (NOT hardware-tested)** である。実機での耐ノイズ性・再同期マージンは今後の実測課題である。

6 Cドライバ実装リファレンス

6.1 初期化・ピン設定

```
#include "pico/stdlib.h"
#include "hardware/pio.h"
#include "hardware/gpio.h"
#include "hardware/dma.h"
#include "bmc_p2p.pio.h"

#define BMC_CLKDIV 3.0f

typedef struct {
    PIO pio;
    uint sm_tx;
    uint offset_tx;
    uint dma_tx_chan;
} bmc_tx_shared_t;

// TX共有コントローラの初期化 (PIO0を使用)
void bmc_tx_shared_init(bmc_tx_shared_t *ctx, PIO pio, uint sm) {
    ctx->pio = pio;
    ctx->sm_tx = sm;
    ctx->offset_tx = pio_add_program(pio, &bmc_tx_program);

    // TX DMA設定 (32bit転送, DREQ_PIOx_TX)
    ctx->dma_tx_chan = dma_claim_unused_channel(true);
    dma_channel_config c = dma_channel_get_default_config(ctx->dma_tx_chan);
    channel_config_set_transfer_data_size(&c, DMA_SIZE_32);
    channel_config_set_read_increment(&c, true);
    channel_config_set_write_increment(&c, false);
    channel_config_set_dreq(&c, pio_get_dreq(pio, sm, true));
    dma_channel_configure(ctx->dma_tx_chan, &c, &pio->txf[sm], NULL, 0, false);
}

// ポートTXピンの初期化 (平時SIO High固定、読み戻しイネーブル)
void bmc_tx_pin_init(uint pin) {
    gpio_set_slew_rate(pin, GPIO_SLEW_RATE_SLOW);
    gpio_set_drive_strength(pin, GPIO_DRIVE_STRENGTH_4MA);
    gpio_put(pin, 1);
    gpio_set_function(pin, GPIO_FUNC_SIO);
    gpio_set_dir(pin, GPIO_OUT);
    gpio_set_input_enabled(pin, true); // PIO0のmov pins, !pins による出力読み戻しを許可
}

// ポートRXステートマシンの初期化 (PIO1専従リスニング + RX DMA直結)
void bmc_rx_port_init(PIO pio, uint sm, uint offset_rx, uint pin_rx, uint dma_rx_chan, uint32_t *rx_buf,
size_t buf_words) {
    gpio_pull_down(pin_rx);
    gpio_set_input_hysteresis_enabled(pin_rx, true);

    pio_sm_config c = bmc_rx_program_get_default_config(offset_rx);
    sm_config_set_in_pins(&c, pin_rx);
```

```

sm_config_set_jump_pin(&c, pin_rx);
sm_config_set_in_shift(&c, false, true, 32); // MSB-first, autopush 32bit
sm_config_set_clkdiv(&c, BMC_CLKDIV);

pio_gpio_init(pio, pin_rx);
pio_sm_set_consecutive_pindirs(pio, sm, pin_rx, 1, false);
pio_sm_init(pio, sm, offset_rx, &c);
pio_sm_exec(pio, sm, pio_encode_mov_not(pio_x, pio_null)); // X = 0xFFFFFFFF
pio_sm_set_enabled(pio, sm, true);

// RX DMA設定: 4ワードFIFOのオーバーランを防ぐためリングバッファへ常時回収
dma_channel_config dc = dma_channel_get_default_config(dma_rx_chan);
channel_config_set_transfer_data_size(&dc, DMA_SIZE_32);
channel_config_set_read_increment(&dc, false);
channel_config_set_write_increment(&dc, true);
channel_config_set_dreq(&dc, pio_get_dreq(pio, sm, false));
dma_channel_configure(dma_rx_chan, &dc, rx_buf, &pio->rxf[sm], buf_words, true);
}

```

6.2 送信プリミティブ（動的 MUX + 協調 Yield）

```

// 送信実行プリミティブ（上位で排他ロック獲得済みであること）
// packet_buf[0] には（データ総ワード数 - 1）を格納し、packet_buf[1..] にフレーム本体を配置すること
void bmc_tx_send_packet_yield(bmc_tx_shared_t *ctx, uint target_tx_pin,
                             const uint32_t *packet_buf, size_t total_dma_words) {
    PIO pio = ctx->pio;
    uint sm = ctx->sm_tx;

    // 1. ピン設定を動的にPIOへ切り替え（INピンも同一ピンにマップして自反転を成立）
    pio_gpio_init(pio, target_tx_pin);
    pio_sm_set_consecutive_pindirs(pio, sm, target_tx_pin, 1, true);

    pio_sm_config c = bmc_tx_program_get_default_config(ctx->offset_tx);
    sm_config_set_set_pins(&c, target_tx_pin, 1);
    sm_config_set_out_pins(&c, target_tx_pin, 1);
    sm_config_set_in_pins(&c, target_tx_pin); // mov pins, !pins 用の自ピン入力マッピング
    sm_config_set_out_shift(&c, false, false, 32);
    sm_config_set_clkdiv(&c, BMC_CLKDIV);
    pio_sm_init(pio, sm, ctx->offset_tx, &c);
    pio_sm_set_enabled(pio, sm, true);

    // 2. 直前のIRQフラグクリア & DMA転送開始（第0ワード=ワードカウンタ, 第1ワード以降=データ）
    pio_interrupt_clear(pio, 0);
    dma_channel_transfer_from_buffer_now(ctx->dma_tx_chan, packet_buf, total_dma_words);

    // 3. パケット物理出力完了（最終ビット送出後の IRQ 0）まで協調 yield
    while (!pio_interrupt_get(pio, 0)) {
        shizuku_yield(); // 他タスクへCPU権を譲渡
    }
    pio_interrupt_clear(pio, 0);

    // 4. TX SM停止 & ピン制御を安全にSIO（High固定）へ復帰
    pio_sm_set_enabled(pio, sm, false);
    gpio_put(target_tx_pin, 1);
    gpio_set_function(target_tx_pin, GPIO_FUNC_SIO);
}

```

7 リンク死活監視・フェイルセーフデーモン

定期周期（例: 10ms）またはバックグラウンド監視タスクにより、各 RX ピンの物理レベルをサンプリングする。

```

#define LINK_DOWN_TIMEOUT_TICKS 10 // 10ms * 10 = 100ms (Low継続で未接続/切断判定)
#define LINK_UP_STABLE_TICKS 2 // 10ms * 2 = 20ms (High継続でリンク確立判定)

typedef struct {
    uint rx_pin;
    uint tx_pin;
    uint32_t low_count;
    uint32_t high_count;
}

```

```

    bool is_link_up;
} bmc_link_monitor_t;

void bmc_link_monitor_tick_10ms(bmc_link_monitor_t *mon) {
    if (!gpio_get(mon->rx_pin)) {
        mon->high_count = 0;
        mon->low_count++;
        if (mon->low_count >= LINK_DOWN_TIMEOUT_TICKS && mon->is_link_up) {
            mon->is_link_up = false;
            gpio_put(mon->tx_pin, 1);
            on_bmc_link_status_changed(mon->rx_pin, false);
        }
    } else {
        mon->low_count = 0;
        mon->high_count++;
        if (!mon->is_link_up && mon->high_count >= LINK_UP_STABLE_TICKS) {
            mon->is_link_up = true;
            gpio_put(mon->tx_pin, 1);
            on_bmc_link_status_changed(mon->rx_pin, true);
        }
    }
}
}

```

8 設計・仕様の訂正履歴および前提の厳密化 (Mandatory Corrections)

本ドキュメントの初版および先行検討において提示された各種の主張・仮説について、実装事実およびハードウェア検証の観点から以下の通り厳密な訂正と分類（認識論的分離）を行う。

8.1 6 大必須訂正事項 (Mandatory Corrections)

項目	旧主張 (Old Claim / 誤認)	訂正された現実 (Corrected Reality)
1. 完全均一ビット幅	【旧主張】TX ビットセルは前半 8cyc/後半 8cyc 厳密同期の完全均一 16 サイクル幅（均等波形）である。	【非均一ビットセル】ワード内のビットは公称 16 サイクル (8cyc/8cyc) であるが、32bit ワード境界 (OSR 再充填とループ判定) では pull block と jmp y-- の実行によりワード末尾セルが 18 サイクル (+2 サイクル伸長) となる。高速版(bmc_p2p_fast.pio)では通常8サイクル、ワード境界 10 サイクル (+2 サイクル)。RX 側は境界エッジ同期のためこの伸長を許容・吸収する。
2. ± 9.83 ns 保証	【旧主張】実効ジッターマージン ± 9.83 ns が物理的・実機環境において保証される。	【机上モデルであり未確立】 ± 9.83 ns は名目値と対称立ち上がりを仮定した机上計算モデル (Calculation/Model) に過ぎず、実機環境 (実デバイス) での保証値としては未確立 (Not established)。PAD 製造ばらつき、線路容量、非対称遷移、高周波反射は実機特有の未知事項である。
3. 18.75 Mbps PIO/PHY 限界	【旧主張】18.75 Mbps (8cyc/bit @ 150MHz) が抵抗レス直結の物理・PIO 命令両面の絶対限界である。	【実験的探索点 (EXPERIMENTAL)】18.75 Mbps は現行の 1 命令/サイクル同期アーキテクチャ下における実験的探索点に過ぎず、チップおよび物理層 (PHY) の絶対的限界を確定・確立したものではない。別命令構成やクロック変更による更なる限界探索の余地が残されている。
4. SLOW+4mA 誤エッジゼロ	【旧主張】GPIO_SLEW_RATE_SLOW + 4mA 駆動によりリングング反射波を平滑化し、誤エッジは物理的にゼロ。	【未検証の仮定】スルーレート制限と弱駆動は反射波の尖頭値を抑制するが、任意のジャンプ線長や浮遊インダクタンスにおいて「誤エッジが完全にゼロ」であることは実機証明されてお

		らず、理論的期待・前提 (Unverified assumption) である。
5. 量産推奨	【旧主張】 9.375 Mbps (または特定レート) が商用・量産推奨レートとして確立されている。	【量産推奨は未確立】 本プロジェクトはブレッドボード上のジャンパ線直結による実験的 PoC 実証であり、量産推奨 (Production recommendation) は未確立 (Not established)。リファクタリング後のファームウェアは実機ハードウェア検証未実施であり、長期環境試験や EMC 耐量評価も行われていない。
6. wait_sop への JMP による ISR フラッシュ誤認	【旧主張】 pio_sm_exec(jmp(wait_sop)) により RX ISR の端数ビットを瞬時にフラッシュしリセット可能。	【JMP は PC 変更のみ】 RP2350 PIO において jmp 命令はプログラムカウンタ (PC) を変更するだけであり、ISR の蓄積ビットや入力シフトカウンタをクリアしない。端数ビットとカウンタを真にフラッシュ・クリアするには mov isr, null (または SM のリセット・再起動) の実行が必須である。

8.2 情報の厳密な 4 区分 (認識論的ステータス)

- ・ (a) ユーザー報告による実機計測値 (USER-REPORTED MEASUREMENTS) :
 - ▶ プラットフォーム: Raspberry Pi Pico 2 W / RP2350 @ 150.0 MHz sysclk、同一 GP0 GP1 ジャンパ線直結環境
 - ▶ 旧シングルレート BMC (3.125 Mbps): 15,251 pkts, 157.4 kB/s, CRC Err: 0, Drops: 0
 - ▶ ハードウェア UART 3.000 Mbps: 11,389 pkts, 162.5 kB/s, CRC Err: 0, Drops: 0, Verify Err: 0
 - ▶ ハードウェア UART 9.375 Mbps: 20,339 pkts, 290.1 kB/s, CRC Err: 0, Drops: 0, Verify Err: 0
- ・ (b) ソースコード／実装から導出された事実 (Code-Derived Facts) :
 - ▶ PIO 命令長: 標準 TX 14/32, RX 17/32; 高速 TX 14/32, RX 18/32
 - ▶ PIO 配置: TX は pio0、RX は pio1 に完全分離
 - ▶ ビット幅: 標準 通常 16cyc/境界 18cyc; 高速 通常 8cyc/境界 10cyc
 - ▶ ワイヤフォーマット: PREAMBLE (0xAAAA, 2B) + SYNC (0x93C7, 2B) + ヘッダ + ペイロード + CRC16 + パディング ($1 \leq P \leq 4$, 先行 0xFF, 末尾 0xFF/0xFE)
 - ▶ 再同期動作: 受信ドライバによるビット単位探索と任意ビットオフセット (0~7 ビット) からの次フレーム自動復帰
 - ▶ レジスタ操作: mov isr, null による ISR およびシフトカウンタのクリア
- ・ (c) 計算／モデル (Calculation / Model) :
 - ▶ 伝送線路パラメータ ($t_d \approx 2.64$ ns, $Z_0 \approx 120\Omega$)、名目サンプリングマージン計算
- ・ (d) 未検証の前提／実機特有の未知事項 (Unverified Assumptions / Device-Only Unknowns) :
 - ▶ リファクタリング後ファームウェアの実機ハードウェア動作 (ホスト単体テスト合格のみ、実機未検証)
 - ▶ 誤エッジゼロ仮説、実機環境におけるジッターマージン保証、量産適格性